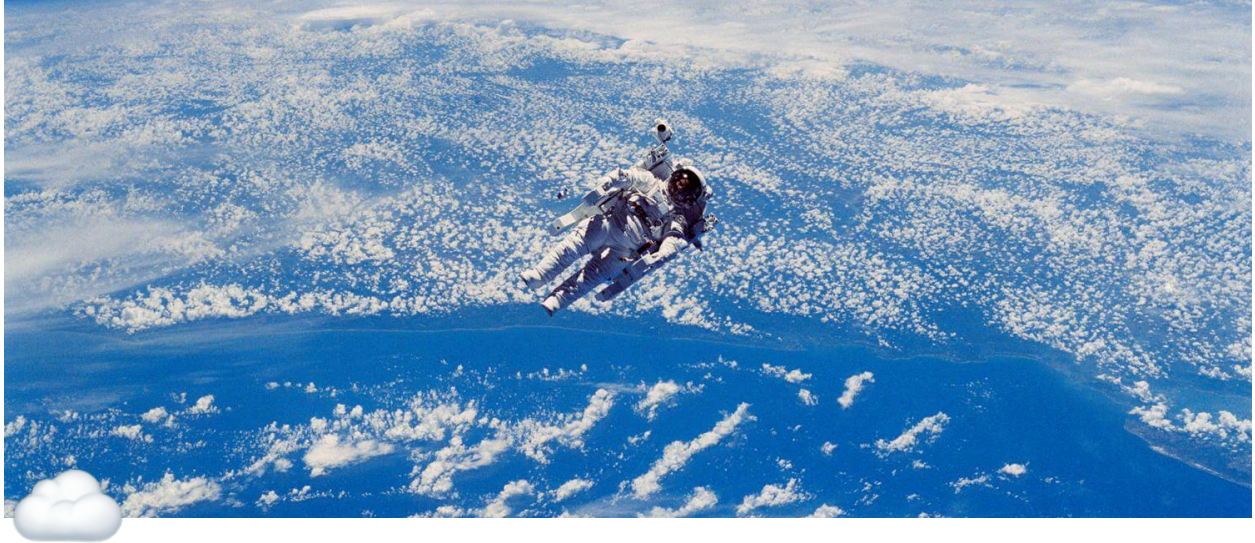




Building an AWS Multi-Account Access Manager: From Setup to Deployment

@October 6, 2025

Why This Project?

Managing access across multiple AWS accounts is something every cloud engineer needs to understand. Whether you're working at a startup that's outgrown a single account or joining an enterprise with dozens of AWS environments, cross-account access is fundamental to AWS security architecture.

I decided to build a Multi-Account Access Manager to learn IAM deeply and create something I could actually use. This post covers the foundational setup: creating multiple AWS accounts, configuring the AWS CLI, and establishing cross-account trust relationships.

Prerequisites

Before starting, I made sure I had:

- A Mac (these instructions are Mac-specific, but the concepts apply everywhere)
- An AWS account
- Basic command line familiarity

Step 1: Installing the Tools

First, I needed to install Homebrew, AWS CLI, Python, and boto3.

Installing Homebrew

Homebrew is a package manager for Mac that makes installing development tools straightforward. I ran:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

The installer prompted for my Mac password and showed what it would install. After installation, it displayed additional commands to add Homebrew to my PATH, which I ran immediately.

Installing AWS CLI and Python

With Homebrew installed, I installed the AWS CLI:

```
brew install awscli
```

Then Python 3:

```
brew install python3
```

And finally boto3, the Python SDK for AWS:

```
pip3 install boto3
```

I verified everything worked:

```
aws --version
```

```
# Output: aws-cli/2.31.8 Python/3.13.7 Darwin/24.6.0 source/arm64
```

```
python3 --version
```

```
# Output: Python 3.12.7
```

Step 2: Creating Multiple AWS Accounts

To demonstrate cross-account access, I needed at least two AWS accounts. I already had my main account, so I created a second one using AWS Organizations.

Using AWS Organizations

AWS Organizations lets you manage multiple accounts under one umbrella, which mirrors how real companies structure their AWS environments.

1. Logged into my main AWS account
2. Navigated to AWS Organizations
3. Clicked "Create organization"
4. Selected "Add an AWS account" then "Create an AWS account"
5. Used the email trick: if my main account was `myemail@gmail.com`, I used `myemail+aws2@gmail.com`
6. Named it "Dev Account"

AWS created the account automatically. My main account ID was `123456789012` and the new account ID was `987654321098`.

Step 3: Configuring AWS CLI Credentials

To interact with AWS programmatically, I needed to configure the AWS CLI with access credentials.

Creating an IAM User

In my main AWS account:

1. Navigated to IAM service
2. Clicked "Users" then "Create user"
3. Named it `cli-user`
4. Attached the `AdministratorAccess` policy (I'll restrict this later)
5. Went to the Security credentials tab
6. Clicked "Create access key"
7. Selected "Command Line Interface (CLI)"
8. Copied the Access Key ID and Secret Access Key

Configuring the CLI

I ran:

```
aws configure
```

It prompted me for:

- Access Key ID (pasted from AWS)
- Secret Access Key (pasted from AWS)
- Region: `us-east-1`
- Output format: (pressed Enter for default)

To verify it worked:

```
aws sts get-caller-identity
```

This returned my user information, confirming authentication was successful.

Step 4: Setting Up Cross-Account Access

This is where IAM really comes into play. I needed to create a role in my second account that my main account could assume.

Accessing the Second Account

Since I created the second account through Organizations, I needed to set up access:

1. Found the account in AWS Organizations and noted its email
2. Opened an incognito browser window
3. Went to AWS sign-in and clicked "Forgot password"
4. Reset the password using the second account's email
5. Logged in with the new password

Creating the Cross-Account Role

In the second account:

1. Navigated to IAM
2. Clicked "Roles" then "Create role"
3. Selected "AWS account" as the trusted entity
4. Selected "Another AWS account"
5. Entered my main account ID: `123456789012`

Error encountered: I initially typed `123455789012` (wrong number). AWS rejected it with the error:

```
Failed to create role CrossAccountAdminRole.  
Invalid principal in policy: "AWS": "123456789012"
```

Fix: I double-checked my main account ID and re-entered it correctly as `123456789012`.

1. Clicked Next and attached the `AdministratorAccess` policy
2. Named the role `CrossAccountAdminRole`
3. Created the role

The resulting ARN was: `arn:aws:iam::987654321098:role/CrossAccountAdminRole`

Step 5: Testing Cross-Account Access

Back in my main account, I tested whether I could assume the role in the second account.

From Terminal:

```
aws sts assume-role \  
  --role-arn arn:aws:iam::987654321098:role/CrossAccountAdminRole \  
  --role-session-name test-session
```

Success! AWS returned temporary credentials including an `AccessKeyId`, `SecretAccessKey`, and `SessionToken`.

This confirmed that:

- The trust relationship was configured correctly
- My main account could assume roles in the second account
- The foundation for building the automation tool was solid

Step 6: Building the Python Tool

With the infrastructure working, I built a Python script to automate the entire process. The script needed to:

- Read configuration about accounts and roles
- Assume roles and retrieve temporary credentials
- Save those credentials for easy use
- Log all access attempts
- Handle errors gracefully

Initial Script Issues

When I first ran the script, I hit an immediate error:

```
ModuleNotFoundError: No module named 'boto3'
```

Fix: I needed to install boto3 in the correct Python environment:

```
pip3 install boto3
```

This installed successfully, though it showed a dependency conflict warning with aiobotocore. The warning didn't affect functionality, so I proceeded.

The Working Script

The complete Python script is available on GitHub: <https://github.com/Joieux/aws-account-manager.git>

Key features of the tool:

- **Configuration Management:** Uses a JSON file to store account information
- **Role Assumption:** Automates the STS AssumeRole process
- **Credential Storage:** Saves temporary credentials to AWS CLI profiles
- **Access Logging:** Tracks all role assumption attempts
- **Error Handling:** Provides specific error messages for common issues
- **Config Validation:** Validates account configuration before use

Testing the Tool

I tested the script with three commands:

1. List accounts:

```
python3 account_manager.py list
```

This created the `accounts.json` config file and displayed both accounts.

2. Check current identity:

```
python3 account_manager.py whoami
```

Showed I was authenticated as cli-user in my main account (123456789012).

3. Assume role in dev account:

```
python3 account_manager.py assume dev
```

This successfully:

- Assumed the CrossAccountAdminRole in my dev account
- Retrieved temporary credentials
- Saved them to a profile called `assumed-dev`
- Logged the access attempt

4. Verify the assumed role:

```
export AWS_PROFILE=assumed-dev  
aws sts get-caller-identity
```

The output confirmed I was now operating as the assumed role in account 093218045525:

```
{  
  "UserId": "AIDAEXAMPLEUSERID123:session-20251006-170405",  
  "Account": "987654321098",  
  "Arn": "arn:aws:sts::987654321098:assumed-role/CrossAccountAdminRole/session-20251006-170405"  
}
```

5. Switch back to main account:

```
unset AWS_PROFILE  
aws sts get-caller-identity
```

And I was back to my main account as cli-user.

Key Lessons Learned

Building this tool taught me several important lessons about AWS, Python, and cloud security:

IAM and Security Concepts

IAM roles are more secure than sharing credentials. Instead of copying access keys between accounts, roles use temporary credentials that expire automatically. This reduces the risk of credential compromise.

Trust relationships are directional. Just because Account A trusts Account B doesn't mean Account B trusts Account A. Each trust relationship must be explicitly configured in the role's trust policy.

Least privilege takes planning. I started with AdministratorAccess to get things working, but in production, I'd need to define specific permissions for each role based on actual requirements.

STS (Security Token Service) is the backbone of temporary credentials. The `AssumeRole` API is what makes cross-account access possible without sharing long-term credentials.

AWS Organizations

Multi-account architecture is the standard. Even small companies use multiple AWS accounts to separate environments (dev, staging, production) and isolate workloads.

Organizations simplifies account management. Using AWS Organizations meant I could manage both accounts from one place and avoid complex billing setups.

The email trick is useful. Using `email+identifier@domain.com` let me create multiple accounts with one email address.

Python and Development

Error handling matters. My first version of the script had generic exception handling. Adding specific exception types (`ClientError`, `NoCredentialsError`) made debugging much easier and provided better user feedback.

Config validation prevents headaches. Checking that the config file has the correct structure before using it saved me from cryptic errors later.

Logging is essential for security tools. The access log provides an audit trail of who assumed what role and when, which is critical for security monitoring.

File operations need care. Managing the AWS credentials file required careful logic to avoid corrupting it or leaving excessive blank lines.

Development Process

Test manually first. Running `aws sts assume-role` from the command line before writing code helped me understand exactly what the API returns and what errors to expect.

Start simple, then improve. I built a basic version that worked, then added error handling, validation, and logging. Trying to build everything at once would have been overwhelming.

Documentation as you go. Writing this blog post while building the tool helped me understand what I was doing and created portfolio content at the same time.

Real errors are learning opportunities. The account ID typo and the boto3 installation issue were frustrating in the moment, but they taught me to double-check inputs and understand Python environments.

What I'd Do Differently

If I were building this for production use, I'd add:

- **Role session duration configuration** - Allow users to specify how long credentials should last
- **MFA support** - Require multi-factor authentication for sensitive role assumptions
- **Credential caching** - Store valid credentials temporarily to avoid unnecessary API calls
- **Better profile management** - Add commands to list active profiles and clean up expired ones
- **More granular permissions** - Replace AdministratorAccess with specific policies for each use case
- **Automated testing** - Write unit tests for the core functionality
- **Better configuration management** - Support environment variables or a config directory

Conclusion

This project gave me hands-on experience with IAM concepts that are fundamental to working with AWS. I now understand:

- How cross-account access works at a technical level
- How to use boto3 to interact with AWS programmatically
- How to structure a Python CLI tool with proper error handling
- How temporary credentials and role assumption work
- Why multi-account architecture is important for security

The tool I built is functional and addresses a real need: easily switching between AWS accounts without manually copying credentials or using the web console. It's something I can actually use in my own work and showcase in my portfolio.

More importantly, I learned by doing. Reading about IAM roles is one thing, but actually setting up trust relationships, debugging permission errors, and handling temporary credentials gave me practical knowledge that will be valuable in any cloud engineering role.

The complete code is available on GitHub: <https://github.com/Joieux/aws-account-manager.git>